

Agent 轨迹检索器的训练与验证

一条可追溯、query 隔离、可独立验收的训练链路

Default Group¹

中国人民大学高瓴人工智能学院，CCIR 参赛队

摘要：传统检索器主要从人的查询、点击和停留日志中学习，而 Agent 会在多轮 Think、Search、Browse 和 Reason 中提出中间 query，并把检索结果作为下一轮推理的观察输入。LRAT 论文说明，Search→Browse 行为、未 Browse 候选和 post-browse reasoning 可以提供与文档效用相关的训练信号。本文围绕官方已发布 training pairs、Agent trajectories、离线 corpus 和原始 Qwen3-Embedding-0.6B，建立一条不重新生成 trajectory、但可追溯、可隔离、可训练、可验收的工程链路。我们回溯 pair 字段并完成 stable/ambiguous/mismatch 审计，按 normalized query 整组切分 train/dev/locked test，采用 1 positive + 5 negatives 的加权对比训练，并用开发集门控学习率与独立冻结验收。最终 A 榜为 Total 40.6、Recall 46.3、Success 21.1、Avg_Steps 16.0；三种 Agent 的现有 B 榜评测记录平均 Total 为 56.18。

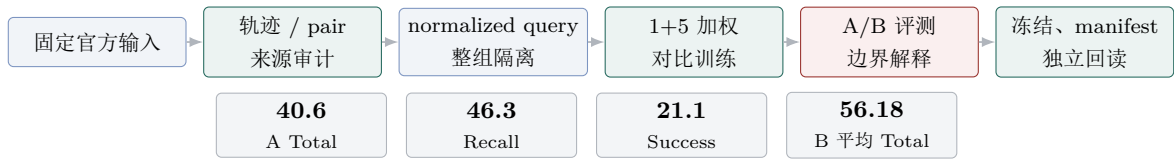


图 1 | 本文的训练与证据链路。改进集中在监督来源、query 隔离、训练控制和独立交付，而不是重新提出 LRAT 的轨迹生成机制。

1 引言

信息检索系统长期围绕人的信息需求设计：用户提交一个相对稳定的 query，系统返回结果页，点击和停留等交互再被用作弱监督。这个假设在 Agentic Search 中发生了变化。深度研究 Agent 会先根据当前 reasoning 产生中间 query，观察 Search 返回的候选摘要，再决定是否 Browse 某篇全文；全文证据又会改变下一轮 reasoning、query 和最终答案。检索结果不再是服务终点，而是 Agent 的感知输入。

这种使用方式带来一个具体的训练问题：一个文档是否“相关”，不能只由它与最终问题的主题相似度决定，还要看它能否推进 Agent 当前的局部推理。若把面向人的通用相关性或未经审计的扁平 pair 直接用于训练，就难以回答三个问题：正负样本为什么成立，同一 query 是否泄漏到验证集，以及一个训练结果能否由独立过程重新加载和回放。

LRAT 将这一问题表述为从 Agent trajectory 学习检索器 [1]。它从 Search→Browse 转移中获取正例候选，把同一候选集内未 Browse 的文档作为经验负例，并用 Browse 后 reasoning 的长度表达相关性强度。本文不把 LRAT 的轨迹生成、LLM judge 或论文实验重新包装成本项目的新贡献；我们的目标是围绕固定官方输入，把这套监督变成一条可审计、可控且可复核的训练链路。

本文的实际贡献如下：

1. 将官方 pair 的 query、正负文档和 `reweight_rate` 逐条回溯到 trajectory 事件和 corpus，并显式区分 stable、ambiguous 和 mismatch。
2. 对 query 做 strip、连续空白合并和 Unicode casefold，按完整 normalized-query group 隔离 train、dev 和 locked test。
3. 固定 1 positive + 5 negatives、128/512 token 预算、官方权重和 cross-device negatives，记录真实训练配置与 loss 归一化方式。
4. 用 dev1500 进行配置门控，随后独立加载、冻结、生成 manifest，并把 A 榜、B 榜和交付状态分开解释。

2 相关工作

面向人的检索训练。传统 dense retriever 和 learning-to-rank 方法通常以人工相关性、点击或其他用户交互作为监督。它们适合“query → 结果页”的服务形式，但这些信号并不直接描述 Agent 在多轮推理中为什么选择某篇全文，也不记录一次检索是否改变了后续行动。

从 Agent 轨迹学习。LRAT 的核心观察是，Agent 的 Search、Browse 和 post-browse reasoning 构成了一种不同于人类点击日志的行为监督。Browse 表示 Agent 愿意读取全文，未 Browse 候选在论文设定中提供了经验负例，而 reasoning length 可以提供强度信息。本文沿用这

套问题语言，用它解释官方 pair 的字段；同时严格区分“论文提出的训练范式”和“本项目在固定官方 pair 上完成的工程实现”。

本文的定位。本项目不是对 LRAT 论文的完整再现，也没有重新运行论文中的 10K query、30B Agent、LLM judge 或 data flywheel。我们研究的是一个受竞赛合同约定的 retriever 构建过程：原始模型、官方 pairs 和推理 contract 是固定输入，改进点是 provenance、query 隔离、训练控制和交付验收。

3 预备知识

3.1 Agent trajectory

设初始任务为 q 。在第 t 轮，Agent 维护 reasoning state r_t ，先产生一个中间 Search query q_t ，检索系统返回候选文档集合 $\mathcal{D}_t = \{d_{t,i}\}_{i=1}^K$ 及摘要。Agent 随后可以选择 Browse 某个候选文档，获得全文证据并把它写回新的 reasoning state。一个简化的交互过程为

Think → Search → Browse → Reason → Answer.

因此，训练信号需要描述“候选文档被如何使用”，而不仅是 query 和文档的静态相似度。

3.2 任务定义与证据边界

本项目的输入是官方已发布的 training pairs、原始 Qwen3-Embedding-0.6B、轨迹与离线 corpus；训练目标是得到一个可独立加载的 dense retriever，并在官方 A/B evaluator contract 下读取 query/document 表示。我们不把 dev1500 当作正式泛化证明，也不使用 locked test 进行学习率或 checkpoint 选择。A 榜是固定 Agent 的端到端证据，B 榜只说明在现有 Agent 评测记录上的可运行性，不能外推为所有 Agent 的普遍保证。

4 数据审计与 query 隔离

4.1 Pair provenance

官方 pair 行包含 query、正例文档引用、负例文档列表和 `reweight_rate`。我们用 `build_provenance.py` 将这些字段回到 Search/Browse/reasoning 事件，再用 `preprocess.py` 检查字段类型、负例 ID 和 corpus 文本一致性。全量 96,504 条 pair 中，96,366 条能够稳定回溯，138 条为 ambiguous，0 条 mismatch；1,989,015 个文档引用和 959,042 个唯一文档 ID 均无缺失或文本不一致。

这里需要保留一个版本边界：LRAT PDF 的一处方法描述写出 91,713 条 pairs，而本地官方发布文件有 96,504 行。两者属于不同发布或处理版本，本文只使用当前项目锁定的 96,504 行，不把两个数字拼成一个训练集，也不据此声称完整复现论文数据规模。

4.2 Query-disjoint split

如果按 source row 随机拆分, 同一 query 的多条 pair 可能同时落入 train 和 dev, dev 指标就会混合记忆重复 query 与真正的 query 适应能力。我们先做去首尾空格、合并连续空白和 Unicode casefold, 再把同一 normalized query 的全部 rows 作为一个 group, 按确定性顺序整组分配。

最终 train/dev/test 的 normalized-query overlap 为 0。Dev 只承担开发与门控职责; locked test 没有用于当前配置选择。这一分割提升的是证据边界的清晰度, 而不是自动保证榜单得分提高。

5 训练方法

5.1 Training sample construction

每条官方 pair 行构成一个 query、一个 positive 和从该行 neg 列表抽取的五个 negatives, 组成六路候选组, positive 固定在组内位置 0。query 和 passage 上限分别为 128 和 512 tokens; 它们是控制六路候选显存与计算量的训练预算, 不被表述为理论无损压缩。榜单推理仍遵守官方 evaluator contract。

5.2 Encoder and weighted contrastive learning

模型从原始 Qwen3-Embedding-0.6B 开始全参数更新, 使用共享 encoder、last-token pooling、L2 normalization 和 1024 维向量。对 query q 和 passage d , 相似度为

$$s(q, d) = \frac{q^T d}{\tau}, \quad \tau = 0.02. \quad (1)$$

设第 j 个 query 的对比交叉熵为 ℓ_j , 官方权重为 w_j , 当前实现的 micro-batch 目标为

$$\mathcal{L} = \frac{\sum_j w_j \ell_j}{\sum_j w_j}. \quad (2)$$

权重改变 query 对总 loss 的贡献, 不改变正负标签。两张 A40 各处理 1 个 query; cross-device gather 后, 另一 query 的候选文档成为 in-batch negatives。gradient accumulation=4 只累积梯度, 不合并四个 micro-step 的候选池, effective query batch 为 8。

5.3 Configuration gate

在固定 dev1500 上比较四个 500-step 学习率点。 2×10^{-6} 同时取得最高 R@1 和 MRR, 才进入完整一轮训练。完整配置为 1 epoch、11,764 optimizer steps、per-device batch 1、accumulation 4、bf16、gradient checkpointing 和 cross-device negatives; 不使用外部语料、教师模型、LoRA 或 Adapter。

6 实验

6.1 Experimental setup

评测使用固定官方 evaluator。训练输入为 94,113 行 query-disjoint train, 基础模型为原始 Qwen3-Embedding-0.6B, 硬件为 $2 \times$ NVIDIA A40。我们保留原始官方 reweight_rate, 不加入外部数据或教师模型。正式结果以 A/B 榜为准, dev 指标只负责配置选择。

6.2 A leaderboard

最终候选相对 Base 提升 12.1 个 Total 点; 在同一 query-disjoint 配方中, 相对 1×10^{-6} 提升 1.1 个 Total 点。相对官方 pairs 一轮, Recall 增加 1.5、Success 增加 0.1, 但 Avg_Steps 增加 0.6, 说明收益并非所有指标单调改善。因而本文把结果解释为一个受控配置在固定 Agent 上的端到端验证, 而不是单一 dev 指标的泛化承诺。

6.3 B leaderboard and delivery

现有 B 榜评测记录中, 三种 Agent 的 Total 为 67.42、60.96 和 40.16, 平均 56.18。该结果支持跨 Agent 可运行性, 但分数差异也说明 Agent 自身的模型能力和行动策略会影响端到端表现, 不能外推为普遍保证。

交付路径遵循“checkpoint $\rightarrow$ freeze + manifest $\rightarrow$ 独立 CPU load/validator $\rightarrow$ HF、GitHub、ZIP”的顺序。公开模型、代码仓库和 reproduction package 的对象身份可以回读; 这些准备与验收不等于自动完成比赛平台上传, 平台提交仍是独立的授权门。

6.4 Limitations and evidence boundary

第一, dev1500 不是 held-out leaderboard, 也不能证明对未知 Agent 的泛化; locked test 未用于当前选择。第二, M10 采用官方已发布 pairs 和当前项目训练配置, 不能写成 LRAT 论文完整实验复现。第三, 训练阶段 128/512 的 truncation contract 与榜单推理 contract 分开; 它控制训练成本, 不等于评测输入被同样截断。第四, A/B 榜回答不同问题: A 榜固定 Agent 的综合效果, B 榜观察多个 Agent 的现有评测表现。

7 结论

本文按照 LRAT 的问题组织方式, 从“Agent 如何使用检索结果”出发, 经过相关工作、轨迹预备知识、监督审计、训练方法和实验边界, 最后回到可复核结论。我们的实际改进不是重新提出一个新的 trajectory mining 范式, 而是把固定官方监督转化为一条可相信的工程链路: 来源可追溯, query 可隔离, 训练可控制, 模型可独立加载, 结果可按 A/B 榜分别解释。

更具体地说, 来源追溯回答监督从哪里来, query-disjoint 回答验证是否泄漏, 1+5 加权训练回答如何在有限预算内保留强度信息, dev gate 和 freeze/manifest 回读

表 1: Query-disjoint 数据合同。

集合	Query groups	Source rows	用途
Train	78,890	94,113	全参数训练
Dev	1,500	–	学习率与配置选择
Locked test	500	–	最终诊断, 不回流选择

表 2: Dev1500 上的短训学习率选择。

Learning rate	R@1	MRR
3×10^{-7}	0.4913	0.6419
5×10^{-7}	0.5073	0.6571
1×10^{-6}	0.5433	0.6831
2×10^{-6}	0.5640	0.6993

表 3: 最终训练配置。

项目	配置
Base model	Qwen3-Embedding-0.6B
Train rows	94,113
GPU / epoch	2×A40 / 1
Optimizer steps	11,764
Batch / accumulation	1 / 4
Group / temperature	6 / 0.02
Query / passage cap	128 / 512
Precision / negatives	bf16 / cross-device

回答结果是否仍是同一个对象。当前证据支持 A 榜 Total 40.6 和 B 榜现有记录平均 56.18; 它同时要求我们保留“不是完整 LRAT 复现、不是普遍 Agent 泛化、不是所有指标均改善”的边界。

A Evidence and reproducibility

本稿正文只保留能帮助理解逻辑的数字; 完整身份和交付证据见项目附录 05_ 复现验收附录.md。当前关键对象包括: 最终 M10 权重 SHA 前缀 cea87cca...a9984, 公开模型 commit 9cdfdba...16bb, canonical GitHub main@31949f6...7bba, 以及 B 榜三字段 JSONL SHA 前缀 02c27117...b3f49。这些摘要用于定位对象, 不替代完整 manifest。

参考文献

- [1] Yuqi Zhou, Sunhao Dai, Changle Qu, Liang Pang, Jun Xu, and Ji-Rong Wen. Learning to Retrieve from Agent Trajectories. Local LRAT paper artifact; official repository: <https://github.com/Yuqi-Zhou/LRAT>.
- [2] Yuqi-Zhou. LRAT training pairs and official project data contract. Hugging Face dataset: <https://huggingface.co/datasets/Yuqi-Zhou/>

表 4: 固定 Agent 的 A 榜端到端结果。

候选	Total ↑	Recall ↑	Success ↑	Avg_Steps ↓
Qwen3-Embedding-0.6B (Base)	28.5	28.5	10.6	17.8
官方 pairs 一轮	40.1	44.8	21.0	15.4
Query-disjoint (1×10^{-6})	39.5	45.0	19.8	15.9
Query-disjoint (2×10^{-6})	40.6	46.3	21.1	16.0

表 5: 可回读证据层。

层级	检查内容与当前边界
输入	manifest 锁定 Base model、raw pairs、provenance、corpus 与 split; 不引入外部语料
加载	新 CPU 进程执行 query/document smoke; 证明对象可加载, 不证明不同硬件训练逐字节一致
训练	回读命令、配置、checkpoint 和 loss 归一化; 证明当前实现可解释, 不等于复现论文全部实验
交付	回读 HF、GitHub、ZIP 的 manifest 与 validator; 证明可复核交付, 不等于比赛平台已上传

LRAT-Train.

- [3] Qwen Team. Qwen3-Embedding-0.6B model card. Hugging Face: <https://huggingface.co/Qwen/Qwen3-Embedding-0.6B>.